

Implementierungsplan (stufenweise)

IDEAL/REAL-Graph als prä-geometrisches offenes Quantensystem

gemäß REAL-Graph_OQS-Definition_v4_rev8

antaris (Requester)

Umsetzung: AI-gestützter Plan

03. Januar 2026

1 Scope und Verbindlichkeit

- **Normative Quelle:** Die Implementierung richtet sich *ausnahmslos* nach der Definition v4_rev8 (Anhang-Dateien).
- **Determinismus:** Es wird ausschließlich **D1 (toleranzbasierter Determinismus)** implementiert. Vollständige Reproduzierbarkeit via Environment-Pinning (D0) bleibt bewusst **außerhalb** des Scopes.
- **Stufenfolge (muss eingehalten werden):** (i) Projektstruktur, (ii) Policy/Determinismus, (iii) Module/Funktionen, (iv) Verdrahtung/Orchestrierung, (v) GUI (Streamlit).
- **Defaultwerte:** Alle in der Definition als Default festgelegten Werte sind *zwingend* als Default in der Implementation zu setzen (z. B. $d_p = 1$, $c_0(e) = 1$, $L_{\text{int}} = 1$, $\eta_{\text{soft}} = 10^{-2}$, ...).

Traceability (Fixierung der Referenz)

Definition (TeX) SHA256	f4cb6b4fcb0a0ff70c49583ef36ab94d4dd9adbf022fd2e22e4eae361ade4e3
Definition (PDF) SHA256	0f926e22fdc9bb2b78b1b490c932ee893d58f63a5369c239ac55d66bfb1e9cee
Modulemap (JSON)	symbol_module_map_v4_rev6.json
Modulemap (SHA256)	e9505a32dacfbf2a8aebf45d1214767e9215220acac1e4f4a667264aa6e3baa7

2 Normative Fixpunkte (Policy, Defaultwerte, Konstanten)

Dieser Abschnitt listet die *verbindlichen* Fixpunkte, die im Code nicht als freie Parameter erscheinen dürfen (nur als Policy/Constant), und die im Plan als eigene Deliverables zu implementieren sind.

2.1 Kanonische Parameterreduktion und Defaults

Größe	Wert	Bemerkung / Quelle
d_p	1 (Default)	Interne Dimension pro Knoten; Default ist normativ.
$c_0(e)$	1	Deterministische Initialisierung aller Leitwerte.
L_{int}	1	Intrinsischer Interaktionsradius/Minimallevel (kanonisch fixiert).
K_{max}	$\max(L_{\text{max}} - 1, 0)$	Skalen-Upper-Bound im Feedback/Update.
α_k	$\frac{r_{\star}^k}{\sum_{j=1}^{K_{\text{max}}} r_{\star}^j}$	Normierte Skalenweights (für $K_{\text{max}} \geq 1$).

Fortsetzung auf der nächsten Seite

Größe	Wert	Bemerkung / Quelle
SG-Fall r_*	$r_{\text{SG}} = \frac{3}{5}$	Fixiert im SG-Fall.
η_{soft}	10^{-2}	Soft-Dämpfung bei Fallback.
$c_{n+1}(e)$	$c_n(e) \exp(\Delta_n(e))$	Provisorischer Update-Schritt vor λ_n -Gauge-Fix.

2.2 D1: Toleranzen und numerische Policy (float64)

- **float64-only:** Alle Kernoperationen (Solve, Cholesky, Eigenwertzerlegung, Exp/Log) sind strikt auf IEEE 754 float64 zu definieren.
- **Maschinenkonstanten:** $\varepsilon_{\text{mach}} := \text{eps}(\text{float64})$, $\varepsilon_{\text{floor}} := \sqrt{\varepsilon_{\text{mach}}}$, $\varepsilon := \varepsilon_{\text{floor}}$.
- **SPD-Regularisierung:** $\beta_{\text{reg}} = 10$, $M_{\text{reg}} = 8$, $\kappa_{\text{max}} = 10^{12}$, deterministische Konditions-/Cholesky-Gates und Eskalationsregel; *COND_FAIL* ist ein Report-/Gate-Feld, kein “valid=0”.
- **D1-Gleichheit:** Für alle numerischen Outputs sind die in der Definition vorgegebenen Toleranzen zu verwenden (z. B. $\tau_R, \tau_d, \tau_{\text{res}}$ für Widerstandsmetrik; $\tau_{\text{rel}} = 10^4 \varepsilon_{\text{floor}}$ etc. für Level-Stabilitäts-Checks; $\tau_{\Delta} = 10^{-10}$ für Δ -Konsistenzcheck).
- **Parallelität:** *nur* zur Map-Phase zulässig; Aggregation/Reduktion muss seriell in kanonischer (lexikographischer) Ordnung erfolgen.

3 Ziel-Artefakte (Repository, Reports, Tests, GUI)

- Python-Paket `realgraph_oqs` mit strikt modularer Struktur (siehe nächste Sektion).
- Deterministische `env_lock.json` (D1): Erfasst Policy-Felder, Versionsstrings, Hashes der Artefakte; keine harte Reproduzierbarkeit garantiert.
- Maschinelles Coverage-Tracking: `symbol_coverage.json` wird aus dem Symbolverzeichnis der Definition extrahiert und pro Symbol als “implemented” gepflegt.
- Test-Suite: Unit-/Property-Tests für alle normativen Definitionen und Gates; Reporting der Pflichtfelder (Fallback-Raten, Validitätsflags, λ -Search-Status, etc.).
- Streamlit-App: Parametersteuerung, Visualisierung (Gewichte vs. induzierte Geometrie), Export aller Artefakte.

4 Projektstruktur (stufe 1)

4.1 Repository-Layout (normativ für die Implementation)

```

repo/
  pyproject.toml
  src/realgraph_oqs/
    __init__.py
    policy/          # D1 numerische Policy, Solve, RegSPD, logdet/expm
    ideal/           # SG/ST Trunk, Wörter, Quotient, B0, Zellen, Patch-1 Ordnung
    algebra/         # Laplacian, Blockzerlegung, Kron/DtN, lokale loopy-Matrizen
    oqs/             # Hilbertraum, CPTP-Kanäle, Choi/J\varepsilon, Mismatch, \kappa
    driver/          # p, p, p, \Delta, $\lambda$-Gauge-Fix, Update cn -> c_{n+1}
    metrics/         # Widerstandsmetrik, Distanz, Embeddings (MDS/harmonisch)
    orchestration/   # Run-Config, Step-Loop, Logging/Reports, Exporte
    viz/            # Plot/Heatmap Utilities (keine Physik, nur Darstellung)

```

```

app/streamlit_app.py
tests/
tools/                # symbol_coverage extractor, schema validators
artifacts/            # Exportziel (JSON/NPZ/PNG/PDF)

```

4.2 Stufe-1 Definition of Done (DoD)

- Importierbares Paket, deterministische Orderings (IDs, Kantenlisten, Zellenlisten).
- Basisschema für Reports: jeder Run erzeugt `run_meta.json` inkl. Policy-Hash, Definition-Hashes, Parameter, Gate-Zustände.
- `symbol_coverage.json` ist im Repo enthalten (initial: implemented=false).

5 Stufenweiser Implementierungsplan (Abschnitts-/Pfeiler-orientiert)

Jede Stufe enthält: **Ziel, Tasks, Artefakte, Tests/DoD**. Referenzen beziehen sich auf die Definition v4_rev8 (Abschnitte/Definitionen).

5.1 Stufe 2: Policy/D1-Determinismus (zuerst, vor allen fachlichen Modulen)

Ziel: Alle numerischen Grundoperationen sind als *deterministische Totalfunktionen* (mit Gates/Reports) verfügbar.

P2.1. Implementiere `policy.float64`: Cast/Checks, $\varepsilon_{\text{mach}}$, $\varepsilon_{\text{floor}}$, ε , $\varepsilon_0 := \max(r_{\star}^{L_{\text{max}}}, \varepsilon_{\text{floor}})$.

P2.2. Implementiere `policy.regspd`: $\text{Sym}(A)$, $\delta_0(A)$, Eskalation $\text{RegSPD}^{(m)}(A) = \text{Sym}(A) + \beta_{\text{reg}}^m \delta_0(A)I$, `cholOK( $\cdot$ )` und Auswahl m_{chol} .

P2.3. Implementiere `policy.solve`: deterministische Solve-Semantik inkl. Gate-Felder `valid`, `COND_FAIL`, Konditionsproxy, Retry-Policy.

P2.4. Implementiere `policy.logdet` und `policy.expm`: deterministische Eigenbasis-Fixierung (“erste nichtverschwindende Komponente positiv”), Eigenwert-Clipping $\lambda_{\text{clip}} := \max(\lambda, \varepsilon_{\text{floor}})$.

P2.5. Implementiere D1-Vergleichsfunktionen (Toleranzen) und Hashing von Artefakten (`sha256`).

Artefakte: `policy/{float64.py, regspd.py, solve.py, logdet.py}`, `env_lock.json`-Schema, `run_meta.json`.

DoD/Tests:

- Determinismus-Regression: identischer Input $\Rightarrow$ identische Outputs innerhalb der D1-Toleranzen.
- Gate-Abdeckung: jede Solve/RegSPD-Operation liefert konsistente Gate-Felder und deterministische Fallbacks.

5.2 Stufe 3: IDEAL-Trunk (Abschnitt 1)

Ziel: Konstruktion von (V, E) bei festem L_{max} , inklusive deterministischer Ordnungen, Rand B_0 , Wort-/Zellstruktur und Patch-1-Subset-Ordnung.

I3.1. Implementiere Alphabet $\mathcal{S}_{\bullet}$, Wortmenge $S_{\bullet}^{\leq L_{\text{max}}}$, deterministische Lex-Ordnung.

I3.2. Implementiere Quotient-Identifikation $\sim_{L_{\text{max}}}$ und Vertex-IDs $[(w, i)]_{\sim}$; liefere deterministische ID-Vergabe.

I3.3. Implementiere Kantenkonstruktion $E = E_{L_{\text{max}}}^{\bullet}$ als einfacher Graph (Loop-Ausschluss).

I3.4. Implementiere B_0 (Corners) und Patch-1-Randordnung $(b_0, \dots, b_{q-1})$.

I3.5. Implementiere Zellenmengen V_w, B_w, I_w sowie die LCP-basierten Zuordnungsgewichte $\mu_{k,w}(e)$ und Supports $W_k(e)$.

I3.6. Kanonischer Zell-Index (für Selektion/GUI, normativ). Erzeuge aus $\{\mathcal{S}_{\bullet}^k\}_{k=0}^{K_{\max}}$ eine vollständige, deterministisch geordnete Zellliste

$$\mathcal{C} := \bigcup_{k=0}^{K_{\max}} \mathcal{S}_{\bullet}^k, \quad \text{Sortierung: } (k, w) \text{ mit } k = |w| \text{ und } w \text{ lexikographisch.}$$

Definiere eine kanonische String-Repräsentation `addr_str` (z. B. $\emptyset \mapsto \text{"EMPTY"}$, $(0, 1, 2) \mapsto \text{"012"}$) sowie die Inversenabbildung `parse_addr`.

I3.7. Adressbaum (für anschauliche Selektion). Definiere für jede Zelle $w \in \mathcal{C}$ den Parent $\pi(w)$ als längstes echtes Präfix (für $w = \emptyset$: $\pi(\emptyset) = \perp$) und exportiere eine Baumstruktur `cells_tree` (Adjazenzliste Parent $\rightarrow$ Children) als reines UI-Artefakt.

I3.8. Markierungs-Mengen für Visualisierung. Implementiere für jedes (k, w) die deterministischen Kanten-/Vertex-Sets

$$E_w^{(k)} := \{e \in E : \mu_{k,w}(e) > 0\}, \quad V_w^{(k)} := \{x \in V : \exists e \in E_w^{(k)} \text{ mit } x \in e\},$$

und stelle eine effiziente Masken-API bereit (`edge_mask(k,w)`, `node_mask(k,w)`), damit IDEAL/REAL-Plots dieselbe Zelle konsistent hervorheben können.

Artefakte: `ideal/sg.py`, `ideal/st.py` (mind. API-kompatibel), `ideal/cells.py`.

DoD/Tests:

- Strukturtests: $|B_0| = q$, E ist loop-less und ohne Multikanten.
- Äquivarianz-Diagnostik (wo implementierbar): Relabel-Run für Corner-Permutationen erzeugt gleiche $\mu_{k,w}$ nach Rückumbenennung.

5.3 Stufe 4: Laplacian, globale/lokale DtN/Kron (Abschnitt 2 und 4.4–4.5)

Ziel: Aus c_n die gewichtete Laplacian L_n sowie globale und lokale DtN-Matrizen deterministisch berechnen.

A4.1. Implementiere $c_0(e) = 1$ und allgemeine $c_n : E \rightarrow (0, \infty)$.

A4.2. Implementiere Laplacian $L(c_n)$ exakt gemäß Definition (Off-Diagonal = $-c_n$, Diagonal = gewichtete Grade).

A4.3. Implementiere globale Kron-Reduktion auf B_0 : Blockzerlegung $V = B_0 \sqcup I$, $(L_n)_{II, \text{reg}} := \text{RegSPD}((L_n)_{II})$, Schur-Komplement Λ_n .

A4.4. Implementiere Randtests $y^{(ij)}$ und Randenergie $E_{B_0,n}^{(ij)}$, $E_{B_0,n}(b_i)$, $E_{B_0,n}^{\text{tot}}$.

A4.5. Implementiere lokales loopy- $Q_{n,w} = (L_n)_{V_w, V_w}$ und lokales DtN/Kron $\Lambda_{n,w}$ auf B_w (mit denselben Policy-Gates).

Artefakte: `algebra/laplacian.py`, `algebra/kron.py`, `algebra/dtn.py`.

DoD/Tests:

- Λ_n symmetrisch (bis numerische Policy), Gate-Felder korrekt.
- Randenergie ist numerisch stabil und deterministisch (D1).

5.4 Stufe 5: OQS-Bausteine, Choi-Regularisierung, Mismatch, κ (Abschnitt 2.4–2.6 und 4.6–4.9)

Ziel: Aus globaler/lokaler Antwort/Kanalstruktur das normativ definierte Mismatch $Mismatch_n(w)$ und den Rückkopplungsfaktor $\kappa_n(w)$ berechnen.

Q5.1. Implementiere Hilbertraum-Konstruktion $H = \ell^2(V) \otimes \mathbb{C}^{d_p}$ und die Support-Reduktionskanäle Φ_B (CPTP) gemäß Definition (direkte Summe, kein ontischer Tensorproduktsplit).

Q5.2. Implementiere die normativen Kanäle $\Phi_{n,w}^{\text{glob}}$ und $\Phi_{n,w}^{\text{loc}}$ (inkl. $J_\varepsilon(\Phi)$ -Mischung mit I/d^2).

Q5.3. Implementiere Umegaki-Entropie $S(\rho||\sigma)$ und das Mismatch

$$Mismatch_n(w) := \frac{S(J_\varepsilon(\Phi_{n,w}^{\text{glob}})||J_\varepsilon(\Phi_{n,w}^{\text{loc}}))}{\log(d^2)} \geq 0.$$

Q5.4. Implementiere Sättigungsfunktion σ (Default `tanh`), $\kappa_n^{\text{raw}}(w) := \sigma(Mismatch_n(w))$, Gates `valid(n,w)`, `validglob(n,w)`, Fallback-Flag $f(n,w)$ und Soft-Fallback $\kappa_n(w)$.

Artefakte: `oqs/hilbert.py`, `oqs/channels.py`, `oqs/choi.py`, `oqs/mismatch.py`.

DoD/Tests:

- $Mismatch_n(w) \geq 0$ numerisch (bis D1), J_ε strikt PD.
- Fallback-Rate-Logging vorhanden: $pf_b(n)$, $pf_{b\text{hard}}(n)$, $pf_{b\text{soft}}(n)$.

5.5 Stufe 6: Zellenergie $\rightarrow p, \tilde{p}$, Update-Exponent Δ , λ -Gauge-Fix (Abschnitt 4.10–4.12)

Ziel: Vollständiges Driver-B-Update $c_n \mapsto c_{n+1}$, inklusive Randenergie-Constraint über deterministische λ_n -Suche.

D6.1. Implementiere harmonische Fortsetzung $u_{n,w}^{(ij)}$ (Solve-Semantik; bei Solve-Fail: Innenpotential $\equiv 0$).

D6.2. Implementiere Gewichte $\omega_{\{i,j\}}(n,w)$ aus Raten-Differenzen $\Delta r_{\{i,j\}}(n,w)$ (inkl. $\varepsilon_{\text{floor}}$ -Stabilisierung und Normierung).

D6.3. Implementiere rohe und gewichtete Zellenergie $E_{n,w}^{\text{raw}}(e)$ und $E_{n,w}(e) = \mu_{k,w}(e)E_{n,w}^{\text{raw}}(e)$.

D6.4. Implementiere $Z_w^{(k)} = \sum_e \mu_{k,w}(e)$, Referenz $\bar{p}_w(e)$ und normierte Verteilung $p_{n,w}(e)$ sowie Zentrierung $\tilde{p}_{n,w}(e) = p_{n,w}(e) - \bar{p}_w(e)$ (inkl. Randfall $w = \emptyset$).

D6.5. Implementiere $\Delta_n(e)$ exakt gemäß Definition, inkl. Konsistenz-Report $\Delta_n^{\text{check}}(e)$ und Pflichtcheck $\max_e |\Delta_n(e) - \Delta_n^{\text{check}}(e)| \leq 10^{-10}$.

D6.6. Implementiere provisorisches Update $\tilde{c}_{n+1}(e) = c_n(e) \exp(\Delta_n(e))$ und die λ -Reskalierung nur auf E_∂ .

D6.7. Implementiere deterministische λ_n -Suche (Bracketing+Bisektion) mit fixierten Budgets ($h_0, \gamma, N_{\text{expand}}, N_{\text{bisect}}$), Energietoleranz T_E , Fallback $\lambda_n := 0$ und Gate-Markierung.

D6.8. Finalisiere $c_{n+1}(e) := c_{n+1}^{(\lambda_n)}(e)$.

Artefakte: `driver/energy.py`, `driver/pdist.py`, `driver/delta.py`, `driver/lambda_gauge.py`, `driver/update.py`.

DoD/Tests:

- Erhaltung $E_{B_0,n+1}^{\text{tot}}(\lambda_n) \approx E_{B_0,n}^{\text{tot}}$ innerhalb T_E ; andernfalls deterministische Invalid-Markierung.
- Alle Pflicht-Logs/Gates werden pro Schritt erzeugt (auch bei Fallbacks).

5.6 Stufe 7: Widerstandsmetrik und Einbettungen (Abschnitt 3)

Ziel: Bestimme deterministisch den effektiven Widerstand $R_n(u, v)$ und die Distanz $d_n(u, v) = \sqrt{R_n(u, v)}$; stelle MDS/harmonische Einbettung als *Visualisierung* bereit.

- M7.1.** Implementiere deterministisches Grounding/Jitter-Verfahren zur Lösung auf reduzierter Laplacian und definiere δ_R analog zur Policy.
- M7.2.** Implementiere $R_n(u, v)$ via Solve-Formulierung (kein implizites “exakte Pseudoinverse“-Erfordernis), inkl. Disconnection $\Rightarrow \infty$.
- M7.3.** Implementiere Distanzmatrix D_n (wo endlich), Validitätsgates und D1-Toleranztests (Dreiecksgleichung als Diagnostik).
- M7.4.** Implementiere Einbettungen: (a) MDS auf D_n ; (b) harmonische Einbettung (wo in der Definition vorgesehen). Beide sind **rein visuell** und dürfen keine Rückwirkung auf die Dynamik haben.

Artefakte: `metrics/resistance.py`, `metrics/mds.py`, `metrics/harmonic_embed.py`.

5.7 Stufe 8: Orchestrierung, Reports, Exporte (Verdrahtung)

Ziel: Ein einheitlicher Run-Driver, der die Stufen 3–7 in eine deterministische Step-Loop integriert, den Audit-Trail vollständig schreibt und reproduzierbare Exporte (inkl. optionaler UI-Artefakte) erzeugt.

- O8.1. RunConfig/RunMeta.** Definiere `RunConfig` mit $(\bullet, L_{\max}, N, d_p)$ sowie allen Policy-/UI-Parametern: Tail-Coarse-Grain (inkl. Methode), Trajectories/Theta, Subsystem-Mode (General/Special) und gewähltes w (Zelle), Update-Scope (derzeit fix: `selected_cell`), Einbettungsmodus (nur Visualisierung) sowie Export-Selektor (UI-Artefakte). Persistiere diese Werte deterministisch in `run_meta.json` und `env_lock.json` (keine UI-abhängigen Seitenefekte).
- O8.2. StepResult + Audit.** Implementiere `StepResult` pro Schritt n : enthält c_n , globale/lokale DtN/Seeds, Mismatch-Maps, κ -Werte, Δ -Beiträge, λ -Gauge-Fix, Gates/Validity-Flags sowie deterministische Hashes. Schreibe pro Run einen `run_report.json` mit:
 - globalem `valid_run`-Flag (AND über Seed-Init + Step-Gates),
 - `errors`-Liste (mindestens `SEED_INIT_INVALID` bei invalidem `Seed_0(w)`),
 - optionalen Diagnose-Zeitreihen (z. B. Mismatch/Delta/Gates) als kompakte Arrays/CSV.
- O8.3. Exports (Core).** Implementiere deterministische Exporte für Kernartefakte (ohne GUI): `npz/npy` (Matrizen), `csv` (Tabellen), `json` (Meta/Audit), `png` (Plots), jeweils mit eindeutigem Namensschema inkl. $L_{\max}$, N , Seed-Hash. Wichtig: keine “stale” Artefakte im Export-Ordner (vor Generierung ggf. leeren).
- O8.4. Exports (UI-Artefakte, optional).** Erzeuge `out/ui_artifacts/` nur, wenn explizit selektiert. Implementiere eine Selektionstabelle (Checkbox-Liste) von UI-Artefakten, gruppiert nach Tabs (Views/Subsystem/Diagnostics/Audit/Tables). Default: *alle deaktiviert*. Nur selektierte Artefakte werden erzeugt und ins ZIP aufgenommen. Insbesondere müssen alle Delta-Artefakte (`delta_hist`, `delta_heat`, `delta_matrix`, `delta_top10`) ebenfalls über diese Selektion steuerbar sein.
- O8.5. D1-Regression.** Implementiere einen Regression-Runner: wiederholte Läufe auf identischem Input erzeugen bit-identische Hashes bzw. Abweichungen nur innerhalb definierter Float64-Toleranzen. Abweichungen werden im Audit-Trail protokolliert.

5.8 Stufe 9: Streamlit GUI (zuletzt)

Ziel: Interaktive App, die den definierten Run-Driver (Stufe 8) benutzt, ohne die Modellphysik zu verändern. Die GUI ist *nicht-normativ* und darf Resultate nicht beeinflussen; sie setzt ausschließlich Konfigurationsparameter und persistiert diese deterministisch im Audit-Trail.

G9.1. Sidebar: vollständige Parameterkontrolle mit Tooltips. Alle Einstellungen erhalten eine kurze `help`-Erklärung (“(?)”). Wesentliche Controls:

- Substratwahl (hier: SG), $L_{\max}$, interne Dimension d_p (Presets),
- **N (Steps) als Integer-Eingabe** (`number_input`), frei wählbar bis **max. 5000**, Schrittwerte 1,
- **Subsystem-Mode: General** (Zellwahl aktiv) vs. **Special** (fix $w = \emptyset$, keine zusätzliche “Außen”-Umwelt innerhalb des Trunks),
- **Tail-Coarse-Grain (B_0):** Toggle (Default wie App), *einmalige* Initialisierung bei $n = 0$; Methode wählbar (`b0_uniform` oder `series_shells`; Default wie App),
- Trajectories/Theta Toggle (Default wie App), Einbettung (MDS vs. harmonisch; nur Visualisierung).

G9.2. Subsystem-Zellwahl (UI + deterministisch). Biete `Mode={General,Special}`: **Special** fixiert $w = \emptyset$ (Trunk-Approximation ab B_0 , keine zusätzliche “Außen”-Umwelt innerhalb des Trunks) und deaktiviert die Auswahl. In **General** zeige eine level-/tiefen-geordnete Tabelle aller Zellen $w \in \mathcal{W}_{K_{\max}}$ mit deterministischer Sortierung (z. B. nach $(k, \text{lex}(w))$). Die Auswahl erfolgt über eine Checkbox-Spalte (Data-Editor) und muss *genau eine* Zeile markieren; ansonsten gilt ein deterministischer Fallback auf die erste Zeile des aktuellen Filters (UI-Warnung, aber deterministisch dokumentiert). Hinweis: Auswahl $w = \emptyset$ in **General** ist semantisch identisch zu **Special**. Persistiere `ui.mode`, `ui.subsystem_cell` (und ggf. Filterparameter) in `run_meta.json` sowie im Report.

G9.3. Markierung in IDEAL/REAL-Visualisierungen. Hebe die ausgewählte Zelle in beiden Views eindeutig hervor (Edge/Node-Overlay). In View A/B darf die Markierung nicht verloren gehen (Regression: “Zellwahl geht ein”).

G9.4. Diagnostics-Tabs. Biete standardisierte Diagnostik: Mismatch-Zeitreihen, Gate-Flags, κ -Werte, sowie Delta-Zeitreihen (Heatmap/Histogramm/Top-10). Diese Artefakte müssen im Export-Selektor verfügbar sein (s. Stufe 8).

G9.5. Export-Tab: selektiver UI-Artefakt-Export. Liste exportierbare Plots/Heatmaps/Tabellen/Matrizen *pro Tab* und mache sie per Checkbox de-/aktivierbar. Default: alle deaktiviert. Beim ZIP-Export werden UI-Artefakte nur erzeugt/inkludiert, wenn selektiert. Vor Erzeugung wird `ui_artifacts/` geleert, um “stale” Inhalte zu vermeiden.

G9.6. Fehler-/Audit-Anzeige. Zeige Seed-Init-Report (inkl. `SEED_INIT_INVALID`) und Step-Gates sichtbar an. Ein invalides Seed muss als Fehler getrackt werden; Fallbacks dürfen nur deterministisch erfolgen und müssen im Audit-Trail stehen.

6 Symbol-Coverage und “exakt gemäß Definition”-Nachweis

6.1 Traceability-Matrix (Definition $\rightarrow$ Code $\rightarrow$ Tests $\rightarrow$ Report)

Die Umsetzung gilt **nur dann** als “exakt gemäß Definition v4_rev8, wenn *jedes* gelabelte normative Element (`\label{<id>}`) in der folgenden Matrix einen eindeutigen Code-Einstiegspunkt, mindestens einen Test sowie ein maschinenlesbares Report-Feld besitzt. Fehlende Zeilen oder leere Felder $\Rightarrow$ FAIL(EXACTNESS).

Def.-Label	Bereich	Code-Entry-Point	Test-ID	Report-Feld
def:saturation	Abs.~4 (Driver)	driver/update.py::saturate_delta	UT:def:saturation	driver.saturation
def:soft_fallback	Abs.~4 (Driver)	driver/fallback.py::apply_fallbacks	UT:def: soft_fallback	driver.fallback
prop: saturation_stability	Abs.~4 (Driver)	driver/update.py::validate_saturation_stability	UT:prop:saturation _stability	checks.saturation_stabil ity
rem:abs_rate	Abs.~4 (Driver)	driver/fallback.py::fallback_rates	UT:rem:abs_rate	acceptance.fallback_rate s
rem:lambda_constraint	Abs.~4 (Driver)	driver/lambda_gate.py::lambda_gate	UT:rem: lambda_constraint	driver.lambda_gate
rem:mismatch_dp	Abs.~4 (Driver)	oqs/mismatch.py::MismatchPolicy	UT:rem:mismatch_dp	policy.dp
rem:tanh_choice	Abs.~4 (Driver)	driver/update.py::tanh_choice_policy	UT:rem:tanh_choice	policy.tanh_choice
rem:umegaki_vs_kl	Abs.~4 (Driver)	oqs/mismatch.py::qre_semantics_note	UT:rem: umegaki_vs_kl	run_meta.scope.qre_note
sec:driverB-equiv	Abs.~4 (Driver)	driver/update.py::driverB_equiv_report	UT:sec: driverB-equiv	driver.equiv
sec:mismatch	Abs.~4 (Driver)	oqs/mismatch.py::compute_mismatch	UT:sec:mismatch	oqs.mismatch
def:coarse_dtn	Abs.~1 (IDEAL)	algebra/dtn_kron.py::coarse_dtn_kron	UT:def:coarse_dtn	algebra.dtn
def: pillarA_bridge_claims	Abs.~1 (IDEAL)	orchestration/run_meta.py::claim_ladder	UT:def:pillarA_br idge_claims	run_meta.claims.bridge
def: pillarA_claim_ladder	Abs.~1 (IDEAL)	orchestration/run_meta.py::claim_ladder	UT:def:pillarA_cl aim_ladder	run_meta.claims.ladder
def:preg_locality	Abs.~1 (IDEAL)	ideal/patch.py::patch1_ordering	UT:def: preg_locality	ideal.patch_meta
def:support_directsum	Abs.~1 (IDEAL)	oqs/hilbert.py::direct_sum_space	UT:def: support_directsum	oqs.hilbert_meta
prop:edges_welldefine d_simple	Abs.~1 (IDEAL)	ideal/trunk.py::validate_simple_graph	UT:prop:edges_well defined_simple	checks.simple_graph
prop:no_tensor_factor	Abs.~1 (IDEAL)	oqs/hilbert.py::assert_no_tensor_api	UT:prop: no_tensor_factor	warnings.no_tensor_facto r
prop:nonlocal_boundary	Abs.~1 (IDEAL)	algebra/dtn_kron.py::boundary_nonlocality_report	UT:prop: nonlocal_boundary	dtn.boundary_nonlocality
prop:padding_injective	Abs.~1 (IDEAL)	ideal/words.py::validate_padding_injective	UT:prop: padding_injective	checks.padding_injective
prop: padding_welldefined	Abs.~1 (IDEAL)	ideal/words.py::pad_word	UT:prop:padding_we lldefined	checks.padding

Fortsetzung auf nächster Seite

Def.-Label	Bereich	Code-Entry-Point	Test-ID	Report-Feld
prop: seed_orbit_invariance	Abs.~1 (IDEAL)	orchestration/seeding.py::validate_seed_orbit_invariance	UT:prop:seed_orbit_invariance	checks.seed_orbit_invariance
rem:approx_locality	Abs.~1 (IDEAL)	orchestration/run_meta.py::scope_flags	UT:rem:approx_locality	run_meta.scope.approx_locality
rem:future_tensor	Abs.~1 (IDEAL)	orchestration/run_meta.py::scope_flags	UT:rem:future_tensor	run_meta.scope.future_tensor
rem:nonlocal_intuition	Abs.~1 (IDEAL)	orchestration/run_meta.py::scope_flags	UT:rem:nonlocal_intuition	run_meta.scope.nonlocal_intuition
rem:partial_trace	Abs.~1 (IDEAL)	oqs/reduction.py::partial_trace_ordered	UT:rem:partial_trace	oqs.partial_trace_meta
rem:pillarA_nonclaims	Abs.~1 (IDEAL)	orchestration/run_meta.py::scope_flags	UT:rem:pillarA_nonclaims	run_meta.scope.nonclaims
rem:preg_energy_term	Abs.~1 (IDEAL)	algebra/laplacian.py::dirichlet_energy	UT:rem:preg_energy_term	algebra.dirichlet_energy
rem:st_separate_verification	Abs.~1 (IDEAL)	ideal/trunk.py::st_disabled_flag	UT:rem:st_separate_verification	run_meta.scope.sg_only
sec:bw_trunk	Abs.~1 (IDEAL)	ideal/trunk.py::build_trunk_sg	UT:sec:bw_trunk	ideal.trunk_meta
sec:det_eff_stoch	Abs.~1 (IDEAL)	orchestration/run_meta.py::determinism_notes	UT:sec:det_eff_stoch	run_meta.det_eff_stoch
sec: pillarA_scope_claims	Abs.~1 (IDEAL)	orchestration/run_meta.py::scope_flags	UT:sec:pillarA_scope_claims	run_meta.scope
def:Solve	Abs.~2 (prä-OQS)	policy/solve.py::solve_spd_policy	UT:def:Solve	gates.solve
def:acceptance	Abs.~2 (prä-OQS)	orchestration/acceptance.py::evaluate_acceptance	IT:def:acceptance	acceptance.*
def:cond_monitor	Abs.~2 (prä-OQS)	policy/solve.py::cond_proxy	UT:def:cond_monitor	gates.cond
def:conv_hyp	Abs.~2 (prä-OQS)	orchestration/convergence.py::hypothesis_status	UT:def:conv_hyp	conv.hypotheses
def:conv_protocol	Abs.~2 (prä-OQS)	orchestration/convergence.py::log_conv_metrics	UT:def:conv_protocol	conv.metrics
def:envlock_manifest	Abs.~2 (prä-OQS)	orchestration/env_lock.py::build_env_lock	UT:def:envlock_manifest	env_lock
def:batch_artifacts	Abs.~2 (prä-OQS)	orchestration/batch.py::write_batch_artifacts	UT:def:batch_artifacts	artifacts.batch_cells
def:expm_det	Abs.~2 (prä-OQS)	policy/expm.py::expm_hermitian	UT:def:expm_det	policy.matrixfunc.expm
def: limit_interpretation	Abs.~2 (prä-OQS)	orchestration/convergence.py::LimitGuard	UT:def:limit_interpretation	conv.limit_guard
def:logm_det	Abs.~2 (prä-OQS)	policy/logdet.py::logdet_psd	UT:def:logm_det	policy.matrixfunc.logdet

Fortsetzung auf nächster Seite

Def.-Label	Bereich	Code-Entry-Point	Test-ID	Report-Feld
def:regspd	Abs.~2 (prä-OQS)	policy/regspd.py::regspd(A,...)	UT:def:regspd	gates.regspd
def:sensitivity	Abs.~2 (prä-OQS)	orchestration/sensitivity.py::run_sensitivity_suite	UT:def:sensitivity	sensitivity.*
prop:dtm_continuity	Abs.~2 (prä-OQS)	algebra/dtm_kron.py::validate_dtm_continuity	UT:prop: dtm_continuity	checks.dtm_continuity
prop:ideal_limit_path	Abs.~2 (prä-OQS)	ideal/trunk.py::validate_limit_path	UT:prop: ideal_limit_path	checks.ideal_limit_path
rem:channel_notation_pregeom	Abs.~2 (prä-OQS)	oqs/channel.py::ChoiChannel	UT:rem:channel_notation_pregeom	oqs.channel_meta
rem:fb_forensics	Abs.~2 (prä-OQS)	driver/fallback.py::FallbackForensics	UT:rem: fb_forensics	acceptance.forensics
rem:log_total	Abs.~2 (prä-OQS)	orchestration/reporting.py::write_report	UT:rem:log_total	artifacts.report_json
rem:no_double_jitter	Abs.~2 (prä-OQS)	policy/regspd.py::assert_no_double_jitter	UT:rem: no_double_jitter	warnings.no_double_jitter
rem:real_limit_status	Abs.~2 (prä-OQS)	orchestration/convergence.py::RealLimitStatus	UT:rem: real_limit_status	conv.real_limit_status
sec:conv_status	Abs.~2 (prä-OQS)	orchestration/convergence.py::ConvStatus	UT:sec:conv_status	conv.status
sec:env_lock	Abs.~2 (prä-OQS)	orchestration/env_lock.py::EnvLockSchema	UT:sec:env_lock	env_lock.schema
sec:matrixfunc_policy	Abs.~2 (prä-OQS)	policy/__init__.py::MatrixFuncPolicy	UT:sec: matrixfunc_policy	env_lock.matrixfunc_policy
sec:perf	Abs.~2 (prä-OQS)	orchestration/perf.py::PerfGuards	UT:sec:perf	perf.*
sec:reg_limit	Abs.~2 (prä-OQS)	policy/regspd.py::JitterPolicy	UT:sec:reg_limit	run_meta.jitter_policy
sec:solve_semantics	Abs.~2 (prä-OQS)	policy/solve.py::SolveReport	UT:sec: solve_semantics	run_meta.solve_semantics
app:params_sec2	App.~(Params)	orchestration/config_defaults.py::sec2_defaults	UT:app:params_sec2	config.defaults.sec2
app:params_sec3	App.~(Params)	orchestration/config_defaults.py::sec3_defaults	UT:app:params_sec3	config.defaults.sec3
app:modulemap	App.~B (Modulemap)	orchestration/modulemap.py::load_module_map	UT:app:modulemap	env_lock.module_map_sha256
rem:deltaR_robust	Abs.~3 (Resist.)	metrics/resistance.py::deltaR_policy	UT:rem: deltaR_robust	policy.deltaR
sec:graph_hash_def	Abs.~3 (Resist.)	orchestration/hashing.py::graph_hash	UT:sec: graph_hash_def	hash.graph
sec:resistance_diagnostics	Abs.~3 (Resist.)	metrics/diagnostics.py::resistance_diagnostics	UT:sec:resistance_diagnostics	metrics.diagnostics
subsec:resistance_norm	Abs.~3 (Resist.)	metrics/resistance.py::effective_resistance	UT:subsec: resistance_norm	metrics.resistance

Fortsetzung auf nächster Seite

Def.-Label	Bereich	Code-Entry-Point	Test-ID	Report-Feld
subsec:viz_gauges	Abs.~3 (Resist.)	viz/gauges.py::build_gauges	UT:subsec: viz_gauges	viz.gauges

6.2 Vollständigkeits-Gates (CI/Pre-Commit; normativ)

Neben Unit-/Integrationstests werden zwei *harte* Vollständigkeits-Gates eingeführt:

- G.1. Traceability-Gate:** Datei `tools/traceability_matrix.(json|yaml)` enthält *alle* Labels aus `REAL-Graph_OQS-Definition_v4_rev8.tex`. Ein Check `tools/check_traceability.py` vergleicht (i) extrahierte Labels aus der Definition (build-time) gegen (ii) die Matrix. Fehlt ein Label oder ist ein Feld leer $\Rightarrow$ `FAIL(EXACTNESS)` und CI bricht ab.
- G.2. Symbol-Coverage-Gate:** `symbol_coverage.json` muss für alle *normativen* Symbole (Status “normativ” oder “normative Defaultwerte”) `module  $\neq$  ”` und `implemented=true` besitzen. Zusätzlich muss mindestens ein Test pro Symbol eine Codepfad-Referenz auslösen (über Coverage-Tagging, siehe `tests/coverage_tags.py`). Fehlt dies $\Rightarrow$ `FAIL(SYMBOL_COVERAGE)`.

Diese Gates sind *Teil der Spezifikations-Compliance* und werden nicht als “Variante” abwählbar gemacht.

6.3 Akzeptanzkriterien aus Def.-Label `def:acceptance` als Definition-of-Done

Die in Def.-Label `def:acceptance` festgelegten Schwellen sind als **Policy-Konstanten** zu implementieren (keine physikalischen Parameter) und in jedem Run zu loggen. Die Umsetzung ist nur dann “exakt”, wenn alle folgenden Reports und Gates implementiert sind:

- **Stabilität/Level-Logging:** $\Delta_L(A)$ (und optional δ_L^{spec}) für jede geloggte Observable.
- **Fensterstartindex** $L_0(A)$: verpflichtend deterministisch als kleinstes L eines akzeptierten Fensters (Definition-Label `def:acceptance`) zu ermitteln und zu loggen; *kein* fest gesetztes L_0 .
- **Globaler Worst-Case** L_0^{glob} : zusätzlich $L_0^{\text{glob}} := \max_{A \in \mathcal{A}_{\text{log}}} L_0(A)$ (bzw. $\perp$) zu berechnen und zu loggen, wobei $\mathcal{A}_{\text{log}}$ die im Run tatsächlich geloggte Observable-Menge ist (Definition-Label `def:acceptance`).
- **Fallback-Rate-Gates:** $p_{\text{fb}}(n)$ sowie $p_{\text{fb,hard}}(n), p_{\text{fb,soft}}(n)$ mit Markierung `RUN_INVALID(FALLBACK_RATE)` bzw. `RUN_INVALID(HARD_FAIL_RATE)` und deterministischem Totalitäts-Fallback $\Delta_n(e) \equiv 0$.
- **RegSPD-/Solve-Gates:** $p_{\text{mreg}}(n)$, Warnung `WARN(REG_ESCALATION)` und Invalidierung `RUN_INVALID(REG_ES)`.
- **λ -Gate:** Budget-Überlauf $\Rightarrow$ `WARN(LAMBDA_BUDGET)` und deterministischer Fallback $\lambda_n := 0$.

6.4 Maschinenlesbares Coverage-Tracking

Im Repo liegt `symbol_coverage.json` (automatisch aus dem Symbolverzeichnis der Definition extrahiert). Die Implementierung gilt nur dann als “exakt gemäß Definition”, wenn:

- jedes Symbol mit *normativer* Zuständigkeit (“normativ”/“normative Defaultwerte”) auf eine Code-Implementierung (`module`) gemappt ist,
- `implemented=true` gesetzt ist *und* mindestens ein Test das Symbol in einem Rechenpfad berührt,
- Abweichungen als **Variante** explizit deklariert sind (sonst: `FAIL`).

6.5 Minimaler Audit-Trail (Pflichtfelder pro Run)

Pro Run müssen mindestens folgende Metadaten vorliegen:

- Definition-Hashes (oben), Policy-Hash, Python/Numpy/Scipy-Versionstrings.
- Parameter: $\bullet, L_{\text{max}}, n_{\text{max}}, d_p$, sowie alle Fix-Policy-Konstanten.

- Gate-Felder: `valid`, `validglob`, `COND_FAIL`, `lambda_bracket_found`, `lambda_fallback`.
- Akzeptanzfelder: $p_{\text{fb}}(n)$, $p_{\text{fb,hard}}(n)$, $p_{\text{fb,soft}}(n)$, $p_{\text{mreg}}(n)$, sowie $L_0(A)$ (oder $\perp$) pro gelogger Observable und zusätzlich L_0^{glob} (oder $\perp$); zusätzlich die in der Definition geforderten Konsistenzchecks (insb. Δ -Check).

7 Kurz-Checkliste für die Umsetzung

- C.1. Kein GUI vor Orchestrierung:** Streamlit ist ein reiner Frontend-Wrapper über `orchestration.run()`.
- C.2. Keine impliziten Freiheiten:** Alle Defaults aus der Definition sind im Code als Defaults gesetzt; freie Parameter sind nur die als “frei” markierten Größen (insb. L_{max} , $\bullet$, optional d_p).
- C.3. Gates statt stille Fehler:** Numerische Probleme sind als deterministische Gates/Reports zu markieren, nicht still zu “reparieren”.
- C.4. D1-only:** Keine Behauptung von bitgenauer Reproduzierbarkeit; nur Toleranz-basierte Regression.